

Regular Language

For every NFA $N = (Q = \{0, \dots, n-1\}, \Sigma, \delta, 0, F)$, there exists an equivalent DFA $D = (Q_D, \Sigma, \delta_D, q'_0, F_D)$, where $q'_0 = q_{\{0\}}$,

$$Q_D = \{q_S : S \subseteq \{0, \dots, n-1\}\}$$

$$\delta_D(q_S, a) = q_{S'}, \quad S' = \bigcup_{p \in S} \delta(p, a)$$

$$F_D = \{q_S : S \cap F \neq \emptyset\}$$

Can u give a DFA with the same number of states as of the underlying NFA (in general)? No.

Let A and B be regular languages, then $\overline{A}$ is regular, $A \cup B$, $A \circ B$, A^* , and $A \cap B$ are all regular.

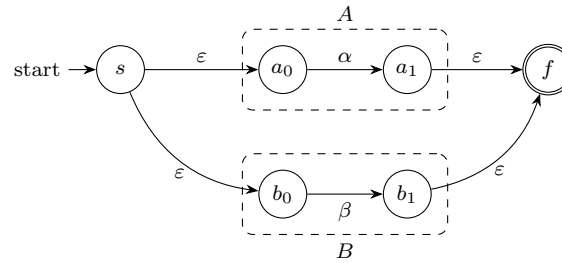
Comment: if u cant see if a language is regular, maybe think about its complement?

Pumping Lemma:

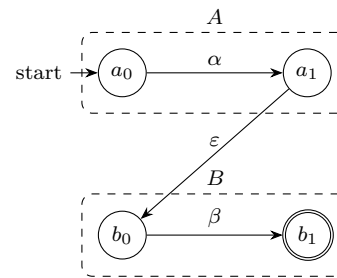
Given regular language L and pumping length p , then for $w = xyz$ length at least p , we have (1) $|y| \geq 1$, (2) $|xy| \leq p$, and (3) $(\forall n \geq 0)(xy^n z \in L)$.

A language is regular if and only if some DFA/ NFA recognizes it if and only if some regex describes it.

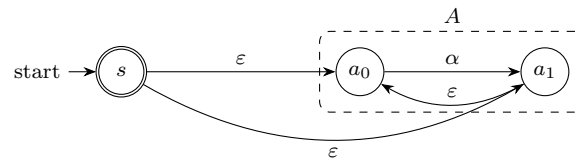
Union:



Concatenation:



Star:



Context Free Language

CFG for $A = \{x : x \text{ has same number of 0s and 1s}\}$:
Grammar $G = (\{S\}, \{0, 1\}, P, S)$ with P being:

$$S \rightarrow 0S1 \mid 1S0 \mid SS \mid \varepsilon$$

CFG for Palindromes: $A = \{w \in \{a, b\}^* : w^R = w\}$:
Grammar $G = (\{S\}, \{0, 1\}, P, S)$ with P being:

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$$

CFGs for $A = \{w \in \{(\cdot)\}^* : w \text{ is balanced}\}$:
Grammar $G_1 : S \rightarrow (S) \mid (SS) \mid SS \mid \varepsilon$.
Grammar $G_2 : S \rightarrow (S)S \mid \varepsilon$.

CFGs for $A = \{a^{2n}b^n : n \geq 0\}$:
Grammar $G : S \rightarrow aaSb \mid \varepsilon$.

Pumping Lemma (CFG):

L being CFL, pumping length p , for all s with $|s| \geq p$, we have (1) $s = uvwxy$, (2) $|vx| > 0$, (3) $|vwx| \leq p$, and (4) $(\forall n \geq 0)(uv^nwx^ny \in L)$.

CFLs are closed under union, concatenation, and star. They are NOT closed under intersection or complementation. However, the intersection of a CFL and a regular is CFL.

Regular Expressions to CFGs

$\emptyset$ can be generated by a CFG with no production rules. a and ε can be generated by a CFG $S \rightarrow a$ and by $S \rightarrow \varepsilon$, respectively. Now suppose R_1 is generated by (V_1, Σ, P_1, S_1) and R_2 is generated by (V_2, Σ, P_2, S_2) , where $V_1 \cap V_2 = \emptyset$.

- $R_1 \cup R_2$ can be generated by $(V_1 \cup V_2 \cup \{S_{\text{new}}\}, \Sigma, P_1 \cup P_2 \cup \{S_{\text{new}} \rightarrow S_1 \mid S_2\}, S_{\text{new}})$, where $S_{\text{new}} \notin V_1 \cup V_2$.
- $R_1 \cdot R_2$ can be generated by $(V_1 \cup V_2 \cup \{S_{\text{new}}\}, \Sigma, P_1 \cup P_2 \cup \{S_{\text{new}} \rightarrow S_1S_2\}, S_{\text{new}})$, where $S_{\text{new}} \notin V_1 \cup V_2$.
- R_1^* can be generated by $(V_1 \cup \{S_{\text{new}}\}, \Sigma, P_1 \cup \{S_{\text{new}} \rightarrow \varepsilon \mid S_1S_{\text{new}}\}, S_{\text{new}})$, where $S_{\text{new}} \notin V_1$.

DFAs to CFGs

We start with a DFA $M = (Q, \Sigma, \delta, q_0, F)$.

- Make a variable R_i for each $q_i \in Q$.
- Add the rule $R_i \rightarrow aR_j$ if $\delta(q_i, a) = q_j$.
- Add the rule $R_i \rightarrow \varepsilon$ if $q_i \in F$.
- The start variable is R_0 .

To prove that this is correct, we use induction, which is left as an exercise.

PDA

PDAs are non-deterministic.

The class of languages accepted by PDAs is exactly CFL.

CFL to PDA

- $\delta(q_{\text{start}}, \epsilon, \epsilon) = \{(q_{\text{loop}}, S\$)\}$
{Place $\$$ and S on the stack}
- $\delta(q_{\text{loop}}, \epsilon, A) = \{(q_{\text{loop}}, u) : A \rightarrow u \text{ is a rule of } G\}$
{select a rule with A on LHS and push RHS onto stack}
- $\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \epsilon)\}$.
{match terminal symbol in input to one in rule}
- $\delta(q_{\text{loop}}, \epsilon, \$) = \{(q_{\text{accept}}, \epsilon)\}$
{accept if stack empty and input read}

PDA to CFL

We may assume that

- Has a single accept state q_{accept}
- Empties its stack before accepting
- Either pushes a symbol onto the stack or pops one off, but not both in one move

Suppose $P = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}})$.

- For every $p, q \in Q$, add a variable A_{pq} which is to generate all strings which take p with **empty stack** to q with **empty stack**.
- Start variable: $A_{q_0q_{\text{accept}}}$.
- Add $A_{pp} \rightarrow \epsilon$ to the production rules for all $p \in Q$.
- Adding productions rules whose lefthand side is A_{pq} for all $p, q \in Q$ (including the case $p = q$)
- Idea:** Any x that takes p with empty stack to q with empty stack, the first move is a **push** and the last is a **pop**. The **last** symbol popped is either the **first** symbol (first case), or not (second case). We will add rules to account for both **first case** and **second case**.

[Case one]: If for $r, s \in Q$ and $a, b \in \Sigma \cup \{\varepsilon\}$:



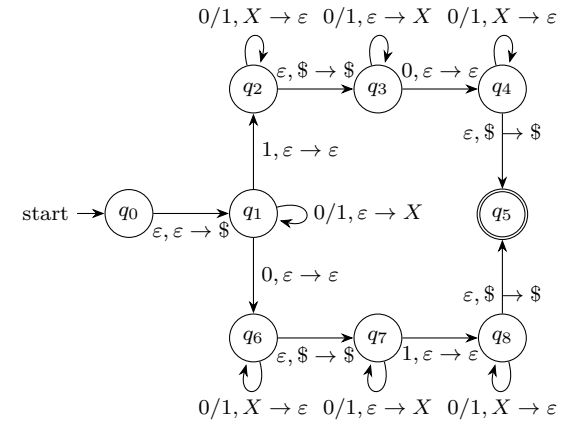
We add

$$A_{pq} \rightarrow aA_{rs}b$$

[Case two]:

$$\text{For all } f \in Q, \text{ add } A_{pq} \rightarrow A_{pf}A_{fq}$$

$A = \{x \in \{0, 1\}^* : x \text{ is not of the form } ww\}$.



This is an example where we exploit the property of PDA being non-deterministic.